

Datenstruktur `Stack` in Java

Merkmal	Beschreibung
Implementierung	Ein <code>Stack</code> basierend auf einem Array speichert Elemente in einer linearen Reihenfolge, wobei das letzte eingefügte Element das erste entfernte ist (LIFO-Prinzip – Last In, First Out).
Datenstruktur	Ein Array wird als interne Speicherstruktur verwendet, um die Elemente des Stacks zu halten. Der Zugriff auf das Array erfolgt über einen Index.
Größe und Kapazität	Der Stack hat eine feste Größe, die beim Erstellen des Arrays festgelegt wird. Wird diese Kapazität überschritten, muss der Stack entweder neu dimensioniert oder es muss ein Fehler ausgelöst werden.
Performance	Die Zeitkomplexität für das Hinzufügen und Entfernen von Elementen ist konstant ($O(1)$), solange keine Neukapazität erforderlich ist.
Thread-Sicherheit	Ein <code>Stack</code> basierend auf einem Array ist nicht thread-sicher. Für parallele Anwendungen sollte ein synchronisierter Stack oder eine <code>ConcurrentLinkedStack</code> verwendet werden.

Methoden

Methode	Beschreibung
<code>push(E e)</code>	Fügt das Element oben auf den Stack hinzu. Bei Erreichen der Kapazitätsgrenze wird die Größe des Arrays oft verdoppelt (automatische Erweiterung).
<code>pop()</code>	Entfernt und gibt das oberste Element des Stacks zurück. Wirft eine <code>EmptyStackException</code> , wenn der Stack leer ist.
<code>peek()</code>	Gibt das oberste Element des Stacks zurück, ohne es zu entfernen. Gibt <code>null</code> oder wirft eine <code>EmptyStackException</code> , wenn der Stack leer ist.
<code>size()</code>	Gibt die Anzahl der Elemente im Stack zurück.
<code>isEmpty()</code>	Gibt <code>true</code> zurück, wenn der Stack leer ist, andernfalls <code>false</code> .
<code>clear()</code>	Entfernt alle Elemente aus dem Stack.
<code>search(Object o)</code>	Sucht nach einem Element im Stack und gibt die Position des Elements vom obersten Element (1 = oberstes Element) zurück. Gibt <code>-1</code> zurück, wenn das Element nicht gefunden wird.